Document Number: P1186R1
Date: 2019-01-22
Audience: EWG
Reply-To: Barry Revzin, barry dot revzin at gmail dot com

When do you actually use `<=>`?

Contents

§ 1. Revision History

R0 of this paper was approved by both EWG and LEWG. Under Core review, the issue of unintentional comparison category strengthening was brought up as a reason to strongly oppose the design. As a result, this revision proposes a different way to solve the issues presented in R0.

The library portion of R0 was moved into P1188R0. This paper is *solely* a proposal for language change.

§ 2. Motivation

P0515 introduced `operator<=>` as a way of generating all six comparison operators from a single function. As a result of P1185R0, that has become two functions, but importantly you still only need to declare one operator function to generate each of the four relational comparison operators.

In a future world, where all types have adopted `<=>`, this will work great. It will be very easy to implement `<=>` for a type like `optional<T>` (writing as a non-member function for clarity):

```
template <typename T>
compare_3way_type_t<T> // see P1188
operator<=>(optional<T> const& lhs, optional<T> const& rhs)
{
    if (lhs.has_value() && rhs.has_value()) {
        return *lhs <=> *rhs;
    } else {
        return lhs.has_value() <=> rhs.has_value();
    }
}
```

This is a clean and elegant way of implementing this functionality, and gives us `<`, `>`, `<=`, and `>=` that all do the right thing. What about `vector<T>`?

```
template <typename T>
compare_3way_type_t<T>
operator<=>(vector<T> const& lhs, vector<T> const& rhs)
{
    return lexicographical_compare_3way(
        lhs.begin(), lhs.end(),
```

```
        rhs.begin(), rhs.end());
}
```

Even better.

What about a simple aggregate type, where all we want is to do normal member-by-member lexicographical comparison? No problem:

```
struct Aggr {
    X x;
    Y y;
    Z z;

    auto operator<=>(Aggr const&) const = default;
};
```

Beautiful.

2.1. An Adoption Story

The problem is that we're not in this future world quite yet. No program-defined types have `<=>`, the only standard library type that has `<=>` so far is `nullptr_t`. Which means we can't just replace the existing relational operators from `optional<T>` and `vector<T>` with `<=>` and probably won't be able to just default `Aggr`'s `<=>`. We need to do something more involved.

How do we implement `<=>` for a type that looks like this:

```
// not in our immedate control
struct Legacy {
    bool operator==(Legacy const&) const;
    bool operator<(Legacy const&) const;
};

// trying to write/update this type
struct Aggr {
    int i;
    char c;
    Legacy q;

    // ok, easy, thanks to P1185
    bool operator==(Aggr const&) const = default;

    // ... but not this
    auto operator<=>(Aggr const&) const = default;
};
```

The implementation of `<=>` won't work for `Aggr`. `Legacy` doesn't have a `<=>`, so our spaceship operator ends up being defined as deleted. We don't get the "free" memberwise comparison from just defaulting. Right now, we have to write it by hand:

```
strong_ordering operator<=>(Aggr const& rhs) const
{
    if (auto cmp = i <=> rhs.i; cmp != 0) return cmp;
    if (auto cmp = c <=> rhs.c; cmp != 0) return cmp;

    if (q == rhs.q) return strong_ordering::equal;
    if (q < rhs.q) return strong_ordering::less;
    return strong_ordering::greater;
}
```

Such an implementation would always give us a correct answer, but it's not actually a good implementation. At some point, `Legacy` is going to adopt `<=>` and we really need to plan in advance for that scenario; we definitely want to use `<=>` whenever it's available.

It would be better to write:

```
strong_ordering operator<=>(Aggr const& rhs) const
{
    if (auto cmp = i <=> rhs.i; cmp != 0) return cmp;
```

```
    if (auto cmp = c <=> rhs.c; cmp != 0) return cmp;
    return compare_3way(q, rhs.q);
}
```

It's at this point that R0 went onto suggest that because `compare_3way()` is transparent to `<=>`, you may as well just always use `compare_3way()` and then you may as well just define `<=>` to be that exact logic. That language change would allow us to just `= default` the spaceship operator for types like `Aggr`.

```
// P1186R0, this involves just synthesizing an <=> for Legacy
auto operator<=>(Aggr const&) const = default;
```

2.2. The Case Against Automatic Synthesis

Consider the following legacy type:

```
struct Q {
    float f;
    bool operator==(Q rhs) const { return f == rhs.f; }
    bool operator<(Q rhs) const { return f < rhs.f; }
    bool operator>(Q rhs) const { return f > rhs.f; }
};
```

Using `float` just makes for a short example, but the salient point here is that `Q`'s ordering is partial, not total. The significance of partial orders is that these can all be `false`:

```
Q{1.0f} == Q{NAN}; // false
Q{1.0f} < Q{NAN};  // false
Q{1.0f} > Q{NAN};  // false
```

However, the proposed synthesis rules in P1186R0 would have led (with no source code changes!) to the following:

```
Q{1.0f} > Q{NAN};       // false
Q{1.0f} <=> Q{NAN} > 0; // true
```

This is because the proposed rules assumed a total order, wherein `!(a == b) && !(a < b)` imply `a > b`.

Now, you might ask... why don't we just synthesize a *partial* ordering instead of a *total* ordering? Wouldn't we get it correct in that situation? Yes, we would. But synthesizing a partial order requires an extra comparison:

```
friend partial_ordering operator<=>(Q const& a, Q const& b)
{
    if (a == b) return partial_ordering::equivalent;
    if (a < b)  return partial_ordering::less;
    if (b < a)  return partial_ordering::greater;
    return partial_ordering::unordered;
}
```

Many types which do not provide `<=>` do still implement a total order. While assuming a partial order is completely safe and correct (we might say `equivalent` when it really should be `equal`, but at least we won't ever say `greater` when it really should be `unordered`!), for many types that's a performance burden. For totally ordered types, that last comparison is unnecessary - since by definition there is no case where we return `unordered`. It would be unfortunate to adopt a language feature as purely a convenience feature to ease adoption of `<=>`, but end up with a feature that many will eschew and hand-write their own comparisons - possibly incorrectly.

The goal of this proposal is to try to have our cake an eat it too:

- allow types like `Aggr` which just want the simple, default, member-wise comparisons to express that with as little typing as possible
- ensure that we do not provide incorrect answers to comparison queries
- ensure that such a feature does not impose overhead over the handwritten equivalent

The first bullet implies the need for *some* language change. The second bullet kills P1186R0, the third bullet kills a variant of P1186R0 that would synthesize `partial_ordering` instead of `strong_ordering`, and the two taken together basically ensure that we cannot have a language feature that synthesis `<=>` for a type with opt-in.

2.3. An Adoption Story for Templates

Taking a step to the side to talk about an adoption story for class templates. How would `vector<T>` and `optional<T>` and similar containers and templates adopt `operator<=>`?

R0 of this paper argued against the claim that "[a]ny compound type should have `<=>` only if all of its constituents have `<=>`." At the time, my understanding of what "conditional spaceship" meant was this:

```
// to handle legacy types. This is called Cpp17LessThanComparable in the
// working draft
template <typename T>
concept HasLess = requires (remove_reference_t<T> const& t) {
    { t < t } -> bool
};

template <HasLess T>
bool operator<(vector<T> const&, vector<T> const&);

template <ThreeWayComparable T> // see P1188
compare_3way_type_t operator<=>(vector<T> const&, vector<T> const&);
```

This is, indeed, a bad implementation strategy because `v1 < v2` would invoke `operator<` even if `operator<=>` was a viable option, so we lose the potential performance benefit. It's quite important to ensure that we use `<=>` if that's at all an option. It's this problem that partially led to my writing P1186R0.

But since I wrote this paper, I've come up with a much better way of conditionally adopting spaceship:

```
template <HasLess T>
bool operator<(vector<T> const&, vector<T> const&);

template <ThreeWayComparable T> requires HasLess<T>
compare_3way_type_t operator<=>(vector<T> const&, vector<T> const&);
```

It's a small, seemingly redundant change (after all, if `ThreeWayComparable<T>` then surely `HasLess<T>` for all types other than pathologically absurd ones that provide `<=>` but explicitly delete `<`), but it ensures that `v1 < v2` invokes `operator<=>` where possible.

Conditionally adopting spaceship between C++17 and C++20 is actually even easier:

```
template <typename T>
enable_if_t<supports_lt<T>::value, bool> // normal C++17 SFINAE machinery
operator<(vector<T> const&, vector<T> const&);

// use the feature-test macro for operator<=>
#if __cpp_impl_three_way_comparison
template <ThreeWayComparable T>
compare_3way_type_t<T> operator<=>(vector<T> const&, vector<T> const&);
#endif
```

In short, conditionally adopting `<=>` has a good user story, once you know how to do it. This is very doable, and is no longer, if of itself, a motivation for making a language change. It is, however, a motivation for *not* synthesizing `<=>` in a way that leads to incorrect answers or poor performance - as this would have far-reaching effects.

The above is solely about the case where we want to adopt `<=>` *conditionally*. If we want to adopt `<=>` *unconditionally*, we'll need to do the same kind of things in the template case as we want to do in the non-template case. We need some way of invoking `<=>` where possible, but falling back to a synthesized three-way comparison from the two-way comparison operators.

2.4. Status Quo

To be perfectly clear, the current rule for defaulting `operator<=>` for a class `C` is roughly as follows:

- For two objects `x` and `y` of type `const C`, we compare their corresponding subobjects x_i and y_i until the first *i* where given `auto` v_i = x_i <=> y_i, v_i != 0. If such an *i* exists, we return v_i. Else, we return `strong_ordering::equal`.
- If the return type of defaulted `operator<=>` is `auto`, we determine the return type by taking the common comparison category of all of the x_i <=> y_i expressions. If the return type is provided, we ensure that it is valid. If any of the pairwise comparisons is ill-formed, or are not

compatible with the provided return type, the defaulted `operator<=>` is defined as deleted.

In other words, for the `Aggr` example, the declaration `strong_ordering operator<=>(Aggr const&) const = default;` expands into something like

```
struct Aggr {
    int i;
    char c;
    Legacy q;

    strong_ordering operator<=>(Aggr const& rhs) const {
        if (auto cmp = i <=> rhs.i; cmp != 0) return cmp;
        if (auto cmp = c <=> rhs.c; cmp != 0) return cmp;
        if (auto cmp = q <=> rhs.q; cmp != 0) return cmp;
        return strong_ordering::equal
    }
};
```

Or it would, if the highlighted line were valid. `Legacy` has no `<=>`, so that pairwise comparison is ill-formed, so the operator function would be defined as deleted.

3. Proposal

This paper proposes a new direction for a stop-gap adoption measure for `operator<=>`: we will synthesize an `operator<=>` for a type, but *only under very specific conditions*, and only when the user provides the comparison category that the comparison needs to use. All we need is a very narrow ability to help with `<=>` adoption. This is that narrow ability.

Currently, the pairwise comparison of the subobjects is always x_i `<=>` y_i. Always `operator<=>`.

This paper proposes defining a new magic specification-only function *3WAY*`<R>(a, b)`, which only has meaning in the context of defining what a defaulted `operator<=>` does. The function definition is very wordy, but it's not actually complicated: we will use the provided return type to synthesize an appropriate ordering. The key points are:

- We will *only* synthesize an ordering if the user provides an explicit return type. We do not synthesize any ordering when the declared return type is `auto`.
- The presence of `<=>` is *always* preferred to any kind of synthetic fallback.
- Synthesizing a `strong_ordering` requires both `==` and `<`.
- Synthesizing a `weak_ordering` can use either `==` and `<` or just `<`.
- Synthesizing a `partial_ordering` requires both `==` and `<` and will do up to three comparisons. Those three comparisons are necessary for correctness. Any fewer comparisons would not be sound.
- Synthesizing either `strong_equality` or `weak_equality` requires `==`.

We then change the meaning of defaulted `operator<=>` to be defined in terms of this magic *3WAY*<R>(x_i, y_i) function (see wording) instead of in terms of x_i `<=>` y_i. If *3WAY*`<R>(a, b)` uses an expression without checking for it and that expression is ill-formed, the function is defined as deleted.

3.1. Soundness of Synthesis

It would be sound to synthesize `strong_ordering` from just performing `<` both ways, but equality is the salient difference between `weak_ordering` and `strong_ordering` and it doesn't seem right to synthesize a `strong_ordering` from a type that doesn't even provide an `==`.

There is no other sound way to synthesize `partial_ordering` from `==` and `<`. If we just do `<` both ways, we'd have to decide between `equivalent` and `unordered` in the case where `!(a < b) && !(b < a)` - the former gets the unordered cases wrong and the latter means our order isn't reflexive..

3.2. Explanatory Examples

This might make more sense with examples.

Source Code	Meaning

`struct Aggr {`
`  int i;`
`  char c;`
	`struct Aggr {`
	`  int i;`
	`  char c;`

```
    Legacy q;

    auto operator<=>(Aggr const&) const = default;
};
```

```
    Legacy q;

    // x.q <=> y.q is ill-formed and we have no return type
    // to guide our synthesis. Hence, deleted
    auto operator<=>(Aggr const&) const = delete;
};
```

```
struct Aggr {
    int i;
    char c;
    Legacy q;

    strong_ordering operator<=>(Aggr const&) const = default;
};
```

```
struct Aggr {
    int i;
    char c;
    Legacy q;

    strong_ordering operator<=>(Aggr const& rhs) const {
        if (auto cmp = i <=> rhs.i; cmp != 0) return cmp;
        if (auto cmp = c <=> rhs.c; cmp != 0) return cmp;

        // synthesizing strong_ordering from == and <
        if (q == rhs.q) return strong_ordering::equal;
        if (q < rhs.q) return strong_ordering::less;

        // sanitizers might also check for
        [[ assert: rhs.q < q; ]]
        return strong_ordering::greater;
    }
};
```

```
struct X {
    bool operator<(X const&) const;
};

struct Y {
    X x;

    strong_ordering operator<=>(Y const&) const = default;
};
```

```
struct X {
    bool operator<(X const&) const;
};

struct Y {
    X x;

    // defined as deleted because X has no <=>, so we fallback
    // to synthesizing from == and <, but we have no ==.
    strong_ordering operator<=>(Y const&) const = delete;
};
```

```
struct W {
    weak_ordering operator<=>(W const&) const;
};

struct Z {
    W w;
    Legacy q;

    strong_ordering operator<=>(Z const&) const = default;
};
```

```
struct W {
    weak_ordering operator<=>(W const&) const;
};

struct Z {
    W w;
    Legacy q;

    // strong_ordering as a return type is not compatible with
    // W's comparison category, which is weak_ordering. Hence
    // defined as deleted
    strong_ordering operator<=>(Z const&) const = delete;
};
```

```
struct W {
    weak_ordering operator<=>(W const&) const;
};

struct Q {
    bool operator<(Q const&) const;
};

struct Z {
    W w;
    Q q;
```

```
struct W {
    weak_ordering operator<=>(W const&) const;
};

struct Q {
    bool operator<(Q const&) const;
};

struct Z {
    W w;
    Q q;

    weak_ordering operator<=>(Z const& rhs) const
    {
```

```
    weak_ordering operator<=>(Z const&) const = default;
};
```

```
    if (auto cmp = w <=> rhs.w; cmp != 0) return cmp;

    // synthesizing weak_ordering from JUST <
    if (q < rhs.q) return weak_ordering::less;
    if (rhs.q < q) return weak_ordering::greater;
    return weak_ordering::equivalent;
  }
};
```

3.3. Differences from Status Quo and P1186R0

Consider the highlighted lines in the following example:

```
struct Q {
    bool operator==(Q const&) const;
    bool operator<(Q const&) const;
};

Q{} <=> Q{}; // #1

struct X {
    Q q;
    auto operator<=>(X const&) const = default; // #2
};

struct Y {
    Q q;
    strong_ordering operator<=>(Y const&) const = default; // #3
};
```

In the working draft, `#1` is ill-formed and `#2` and `#3` are both defined as deleted because `Q` has no `<=>`.

With P1186R0, `#1` is a valid expression of type `std::strong_ordering`, and `#2` and `#3` are both defined as defaulted. In all cases, synthesizing a strong comparison.

With this proposal, `#1` is *still* ill-formed. `#2` is defined as deleted, because `Q` still has no `<=>`. The only change is that in the case of `#3`, because we know the user wants `strong_ordering`, we provide one.

3.4. Building complexity

The proposal here *only* applies to the specific case where we are defaulting `operator<=>` and provide the comparison category that we want to default to. That might seem inherently limiting, but we can build up quite a lot from there.

Consider `std::pair<T, U>`. Today, its `operator<=` is defined in terms of its `operator<`, which assumes a weak ordering. One thing we could do (which this paper is not proposing, this is just a thought experiment) is to synthesize `<=>` with weak ordering as a fallback.

We do that with just a simple helper trait (which this paper is also not proposing):

```
// use whatever <=> does, or pick weak_ordering
template <typename T, typename C>
using fallback_to = conditional_t<ThreeWayComparable<T>, compare_3way_type_t<T>, C>;

// and then we can just...
template <typename T, typename U>
struct pair {
    T first;
    U second;

    common_comparison_category_t<
        fallback_to<T, weak_ordering>,
        fallback_to<U, weak_ordering>>
    operator<=>(pair const&) const = default;
};
```

`pair<T,U>` is a simple type, we just want the default comparisons. Being able to default spaceship is precisely what we want. This proposal gets us there, with minimal acrobatics. Note that as a result of P1185R0, this would also give us a defaulted `==`, and hence we get all six comparison functions

in one go.

Building on this idea, we can create a wrapper type which defaults `<=>` using these language rules for a single type, and wrap that into more complex function objects:

```
// a type that defaults a 3-way comparison for T for the given category
template <typename T, typename Cat>
struct cmp_with_fallback {
    T const& t;
    fallback_to<T,Cat> operator<=>(cmp_with_fallback const&) const = default;
};

// Check if that wrapper type has a non-deleted <=>, whether because T
// has one or because T provides the necessary operators for one to be
// synthesized per this proposal
template <typename T, typename Cat>
concept FallbackThreeWayComparable =
    ThreeWayComparable<cmp_with_fallback<T, Cat>>;

// Function objects to do a three-way comparison with the specified fallback
template <typename Cat>
struct compare_3way_fallback_t {
    template <FallbackThreeWayComparable<Cat> T>
    constexpr auto operator()(T const& lhs, T const& rhs) {
        using C = cmp_with_fallback<T, Cat>;
        return C{lhs} <=> C{rhs};
    }
};

template <typename Cat>
inline constexpr compare_3way_fallback_t<Cat> compare_3way_fallback{};
```

And now implementing `<=>` for `vector<T>` unconditionally is straightforward:

```
template <FallbackThreeWayComparable<weak_ordering> T>
constexpr auto operator<=>(vector<T> const& lhs, vector<T> const& rhs) {
    // Use <=> if T has it, otherwise use a combination of either ==/<
    // or just < based on what T actually has. The proposed language
    // change does the right thing for us
    return lexicographical_compare_3way(
        lhs.begin(), lhs.end(),
        rhs.begin(), rhs.end(),
        compare_3way_fallback<weak_ordering>);
}
```

As currently specified, `std::weak_order()` and `std::partial_order()` from [cmp.alg] basically follow the language rules proposed here. We can implement those with a slightly different approach to the above - no fallback necessary here because we need to enforce a particular category:

```
template <typename T, typename Cat>
struct compare_as {
    T const& t;
    Cat operator<=>(compare_as const&) const = default;
};

// Check if the compare_as wrapper has non-deleted <=>, whether because T
// provides the desired comparison category or because we can synthesize one
template <typename T, typename Cat>
concept SyntheticThreeWayComparable = ThreeWayComparable<compare_as<T, Cat>, Cat>;

template <SyntheticThreeWayComparable<weak_ordering> T>
weak_ordering weak_order(T const& a, T const& b) {
    using C = compare_as<T, weak_ordering>;
    return C{a} <=> C{b};
}

template <SyntheticThreeWayComparable<partial_ordering> T>
partial_ordering partial_order(T const& a, T const& b) {
```

```
    using C = compare_as<T, partial_ordering>;
    return C{a} <=> C{b};
}
```

None of the above is being proposed, it's just a demonstration that this language feature is sufficient to build up fairly complex tools in a short amount of code.

3.5. What about `compare_3way()`?

Notably absent from this paper has been a real discussion over the fate of `std::compare_3way()`. R0 of this paper made this algorithm obsolete, but that's technically no longer true. It does, however, fall out from the tools we will need to build up in code to solve other problems. In fact, we've already written it:

```
constexpr inline auto compare_3way = compare_3way_fallback<strong_ordering>;
```

For further discussion, see P1188R0. This paper focuses just on the language change for `operator<=>`.

3.6. What about `XXX_equality`?

This paper proposes synthesizing `strong_equality` and `weak_equality` orderings, simply for consistency, even if such return types from `operator<=>` are somewhat questionable. As long as we have language types for which `<=>` yields a comparison category of type `XXX_equality`, all the rules we build on top of `<=>` should respect that and be consistent.

4. Wording

Remove a sentence from 10.10.2 [class.spaceship], paragraph 1:

> Let x_i be an lvalue denoting the ith element in the expanded list of subobjects for an object x (of length n), where x_i is formed by a sequence of derived-to-base conversions ([over.best.ics]), class member access expressions ([expr.ref]), and array subscript expressions ([expr.sub]) applied to x. The type of the expression x_i `<=>` x_i is denoted by ~~R_i~~ S_i. If the expression is ill-formed, S_i is `void`. It is unspecified whether virtual base class subobjects are compared more than once.

Insert a new paragraph after 10.10.2 [class.spaceship], paragraph 1:

> Define *3WAY*`<R>(a, b)` as follows:
>
> - If `a <=> b` is well-formed and convertible to `R`, `a <=> b`;
> - Otherwise, if `a <=> b` is well-formed, *3WAY*`<R>(a, b)` is ill-formed;
> - Otherwise, if `R` is `strong_ordering`, then `(a == b) ? strong_ordering::equal : ((a < b) ? strong_ordering::less : strong_ordering::greater)`;
> - Otherwise, if `R` is `weak_ordering`, then:
>   - If `a == b` is well-formed and convertible to `bool`, then `(a == b) ? weak_ordering::equivalent : ((a < b) ? weak_ordering::less : weak_ordering::greater)`;
>   - Otherwise, `(a < b) ? weak_ordering::less : ((b < a) ? weak_ordering::greater : weak_ordering::equivalent)`;
> - Otherwise, if `R` is `partial_ordering`, then `(a == b) ? partial_ordering::equivalent : ((a < b) ? partial_ordering::less : ((b < a) ? partial_ordering::greater : partial_ordering::unordered))`;
> - Otherwise, if `R` is `strong_equality`, then `(a == b) ? strong_equality::equal : strong_equality::nonequal`;
> - Otherwise, if `R` is `weak_equality`, then `(a == b) ? weak_equality::equivalent : weak_equality::nonequivalent`;
> - Otherwise, *3WAY*`<R>(a, b)` is ill-formed.

Change 10.10.2 [class.spaceship], paragraph 2 (note that we do *not* want to make the noted case ill-formed, we just want to delete the operator):

> If the declared return type of a defaulted three-way comparison operator function is `auto`, then the return type is deduced as the common comparison type (see below) of S_0, S_1, ..., S_{n-1}. ~~R_0, R_1, ..., R_{n-1} [Note: Otherwise, the program will be ill-formed if the expression x_i <=> x_i is not implicitly convertible to the declared return type for any i. — end note]~~ If the return type is deduced as `void`, the operator function is defined as deleted.
>
> If the declared return type of a defaulted three-way comparison operator function is `R` and any *3WAY*`<R>(`x_i`,`x_i`)` is ill-formed, the operator function is defined as deleted.

Change 10.10.2 [class.spaceship], paragraph 3, to use `3WAY` instead of `<=>`

> The return value `V` of type `R` of the defaulted three-way comparison operator function with parameters `x` and `y` of the same type is determined by comparing corresponding elements x_i and y_i in the expanded lists of subobjects for `x` and `y` until the first index `i` where ~~$x_i <=> y_i$~~ `3WAY<R>(`x_i`, `y_i`)` yields a result value v_i where v_i `!= 0`, contextually converted to `bool`, yields `true`; `V` is v_i converted to `R`. If no such index exists, `V` is `std::strong_ordering::equal` converted to `R`.

§ 5. Acknowledgments

Thanks to Gašper Ažman, Agustín Bergé, Richard Smith, Jeff Snyder, Tim Song, Herb Sutter, and Tony van Eerd for the many discussions around these issues. Thanks to the Core Working Group for being vigilant and ensuring a better proposal.

§ 6. References

- [P0515R3] "Consistent comparison" by Herb Sutter, Jens Maurer, Walter E. Brown, 2017-11-10
- [P1185R0] "<=> != ==" by Barry Revzin, 2018-10-07
- [P1186R0] "When do you actually use <=>?" by Barry Revzin, 2018-10-07
- [P1188R0] "Library utilities for <=>" by Barry Revzin, 2019-01-20
- [revzin.sometimes] "Conditionally implementing spaceship" by Barry Revzin, 2018-12-21